CS3301 DATA STRUCTURES

UNIT I – INTRODUCTION AND FUNDAMENTAL DATA STRUCTURES

Overview of algorithms and real-world applications. Time and space complexity analysis. Arrays, Strings, Structures – real-world use cases (e.g., text processing, sensors), Linked Lists – single, double, circular, Stack – expression evaluation, syntax checking, Queue – circular queue, priority queue, applications in scheduling Recursion Vs Iteration, Real-world use cases: Balanced parentheses, Undo-redo systems, and task queue management

1.1 OVERVIEW OF ALGORITHMS AND REAL-WORLD APPLICATIONS.

- Algorithms are systematic, step-by-step procedures used to solve problems or perform tasks.
- They are essential in various real-world applications.

1. Sorting Algorithms

Sorting algorithms arrange data in a particular order (ascending or descending).

Examples:

- Bubble Sort
- Merge Sort
- Quick Sort
- Insertion Sort

Real-World Applications:

- **E-commerce**: Sorting products by price, rating, or relevance.
- **Databases**: Organizing records for fast retrieval and querying.
- **Search engines**: Sorting search results based on relevance or ranking algorithms.

2. Search Algorithms

These algorithms are used to find specific data within a collection.

Examples:

- **Linear Search**
- **Binary Search**
- **Breadth-First Search (BFS)**

- **Depth-First Search (DFS)**

Real-World Applications:

Google Search: Binary search is used to quickly find relevant results. Navigation Systems: BFS and DFS are used to find the shortest or most optimal routes in GPS apps.

Social Networks: BFS can be used to recommend friends based on connections.

3. Graph Algorithms

Graph algorithms are used to analyze and manipulate graphs, where nodes represent entities and edges represent connections.

Examples:

- Dijkstra's Algorithm (Shortest Path)
- A Algorithm* (Pathfinding)
- Kruskal's and Prim's Algorithms (Minimum Spanning Tree)
- PageRank (Used by Google)

Real-World Applications:

Social Networks: Finding mutual connections or groups of similar people (community detection).

Routing and Network Optimization: Used in telecommunication and internet routing to find the fastest path for data.

AI/Robotics: Pathfinding for autonomous robots and drones.

Other Algorithms Used

- Dynamic Programming
- Greedy Algorithms
- Backtracking Algorithms
- Divide and Conquer Algorithms

Real-World Applications:

- E-commerce: Sorting, search, recommendation systems (Amazon, eBay).
- Social Networks: Graph algorithms, BFS/DFS for friend suggestions (Facebook, LinkedIn).

- Finance: Dynamic programming, greedy algorithms for investment strategies.

- Telecommunications: Graph algorithms for network routing and optimization.

INTRODUCTION

Data Structures Definition

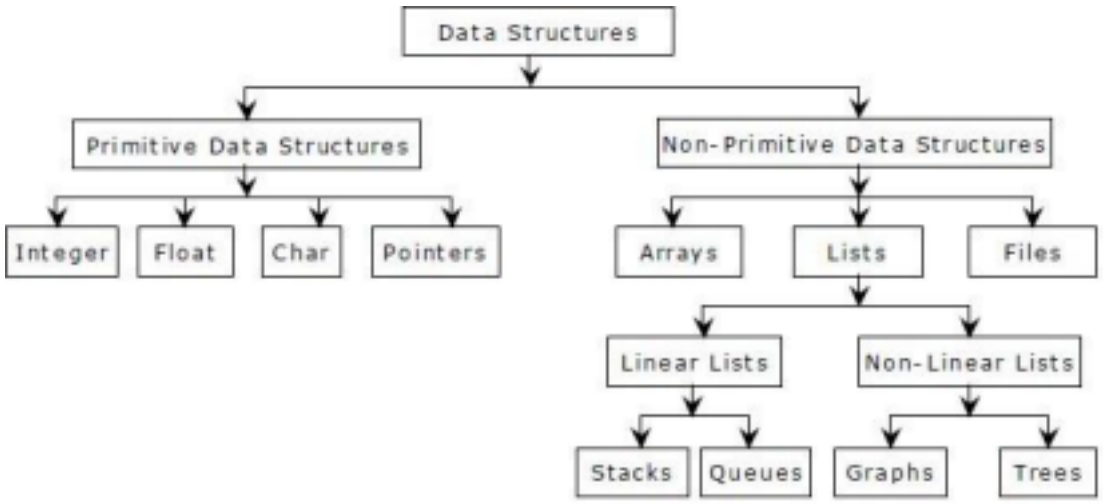
Data Structure is a way of organizing and retrieving data in such a way that we can perform operations on these data in an effective way.

Applications of Data Structures

- Compiler design
- Operating system
- Statistical analysis package
- DBMS
- Numerical analysis
- Simulation
- Artificial Intelligence

CLASSIFICATION OF DATA STRUCTURES

Data structures are generally categorized into two classes: primitive and non primitive data structures.



Primitive data structures are the fundamental data types which are supported by a programming language. Some basic data types are integer, real, character, and boolean.

Non-primitive data structures are those data structures which are created using primitive data structures. Examples of such data structures include linked lists, stacks, trees, and graphs. Non-primitive data structures can further be classified into two

categories: linear and non-linear data structures.

LINEAR AND NON-LINEAR STRUCTURES

LINEAR DATA STRUCTURES

If the elements of a data structure are stored in a linear or sequential order, then it is a linear data structure. Examples include arrays, linked lists, stacks, and queues.

NON-LINEAR DATA STRUCTURES

If the elements of a data structure are not stored in a sequential order, then it is a non linear data structure. The relationship of adjacency is not maintained between elements of a non-linear data structure. Examples include trees and graphs.

OPERATIONS ON DATA STRUCTURES

Traversal: Visit every part of the data structure.

Search: Traversal through the data structure for a given element.

Insertion: Adding new elements to the data structure.

Deletion: Removing an element from the data structure.

Sorting: Rearranging the elements in some type of order (e.g Increasing or Decreasing).

Merging: Combining two similar data structures into one.

1.2 TIME AND SPACE COMPLEXITY ANALYSIS

Time complexity is a type of computational complexity that describes the time required to execute an algorithm. The time complexity of an algorithm is the amount of time it takes for each statement to complete.

What Is Space Complexity?

When an algorithm is run on a computer, it necessitates a certain amount of memory space. The amount of memory used by a program to execute it is represented by its space complexity

What Are Asymptotic Notations?

Asymptotic Notations are programming languages that allow you to analyze an algorithm's running time by identifying its behavior as its input size grows.

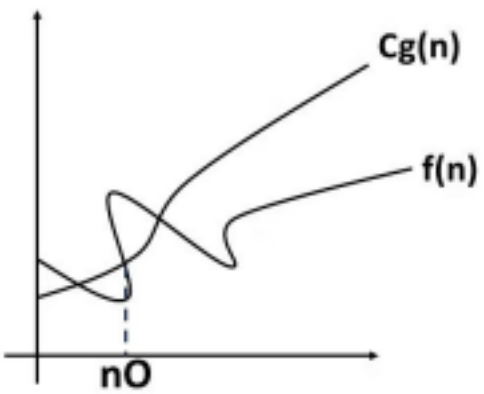
Asymptotic notations are classified into three types:

- Big-Oh (O) notation
- Big Omega (Ω) notation

- Big Theta (Θ) notation

1. Big-Oh (O) notation

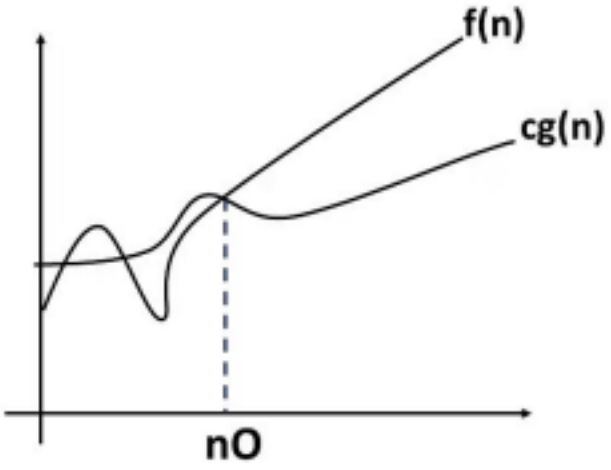
It is the most widely used notation for Asymptotic analysis. It specifies the upper bound of a function, i.e., the maximum time required by an algorithm or the worst-case time complexity. In other words, it returns the highest possible output value (big-O) for a given input.



f(n) = O(g(n))

2. Big-Omega (Ω) notation

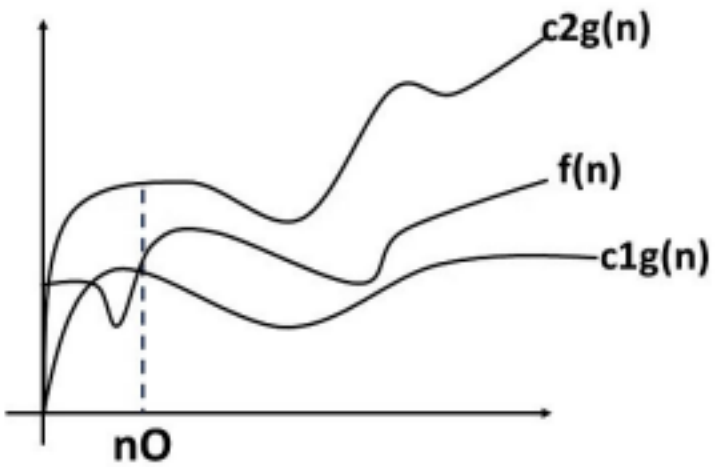
Big-Omega is an Asymptotic Notation for the best case or a floor growth rate for a given function. It gives you an asymptotic lower bound on the growth rate of an algorithm's runtime.



$f(n) = \Omega(g(n))$

3. Big-Theta (Θ) notation

Big theta defines a function's lower and upper bounds, i.e., it exists as both, most, and least boundaries for a given input value.



$f(n) = \theta(g(n))$

1.3 ARRAYS, STRINGS, STRUCTURES

1. Arrays

Definition:

An array is a collection of elements of the same data type, stored in contiguous memory locations.

Key Points:

- Index-based access → fast (O(1)) to read or write by index.

• Fixed size in most languages (C/C++), but dynamic in some (Python lists). • Homogeneous → all elements have the same type.

• Stored sequentially in memory.

Example in C:

```
int arr[5] = {10, 20, 30, 40, 50};
printf("%d", arr[2]); // prints 30
```


Advantages:

• Fast access using index.

• Easy to iterate.

Disadvantages:

• Fixed size (no resizing in static arrays).

• Insertion/deletion in the middle is costly (O(n)).

Common Operations & Time Complexity:

Operation	Time Complexity
Access	O(1)
Search	O(n)
Insertion	O(n)
Deletion	O(n)

2. Strings

Definition:

A string is a sequence of characters stored in contiguous memory, often terminated by a special character (null `'\0'` in C).

Key Points:

• Can be seen as a character array with additional rules.

• Strings are immutable in some languages (Java, Python), meaning they can't be changed after creation.

• In C, strings are mutable and stored as char arrays.

Example in C:

```
char str[] = "Hello";
printf("%s", str); // prints Hello
```


Common String Operations & Time Complexity:

Operation	Time Complexity
Length finding	O(n)
Concatenation	O(n)
Comparison	O(n)
Access by index	O(1)

Uses:

- Text processing
- Searching patterns
- Communication protocols

3. Structures

Definition:

A structure is a user-defined data type in C/C++ (similar to a record in other languages) that groups together different types of data under one name.

Key Points:

- Heterogeneous → can hold integers, floats, strings, etc.
- Allows grouping logically related data together.
- Each member can be of different data type.

Example in C:

```
struct Student {
```

```
 char name[50];
 int roll;
 float marks;
};
struct Student s1 = {"Meena", 101, 95.5};
printf("%s %d %.2f", s1.name, s1.roll, s1.marks);
```


Advantages:

- Groups different types logically.
- Improves code readability.

Disadvantages:

- Access requires the `.` or `->` operator.
- Memory layout may have padding for alignment.

Use Cases:

- Representing a database record.
- Storing complex objects like points, employees, books, etc.

1.4 REAL-WORLD USE CASES (E.G., TEXT PROCESSING, SENSORS)

1. Arrays

Text Processing:

- Used to store sequences of characters (strings).
- Example: Word processors (MS Word, Google Docs) store text as arrays for fast indexing and editing.

Sensors:

- Continuous readings like temperature, heart rate, or motion are stored in arrays.
- Example: A weather station storing hourly temperature readings.

2. Strings

Text Processing:

• Pattern matching for spell-checkers, plagiarism detection, and search engines. • Example: Google search → matches user query (string) with billions of stored documents.

Biological Sensors:

• DNA sequencing → each DNA sequence is a string of nucleotides (A, T, G, C).

1.5 LINKED LISTS

List is an ordered set of elements. The general form of the list is

$$A_0, A_1, A_2, \ldots, A_{N-1}$$

A_0 – First element of the list A_N

$_1$ - Last element of the list

N - Size of the list

If the element at position i is $A_i$then its successor is A_{i+1} and its predecessor is A_{i-1}.

Various operations performed on list

1. Insert (X, 5) - Insert the element X after the position 5.

2. Delete (X) - The element X is deleted

3. Find (X) - Returns the position of X.

4. Print list - Contents of the list is displayed.

Implementation of List ADT

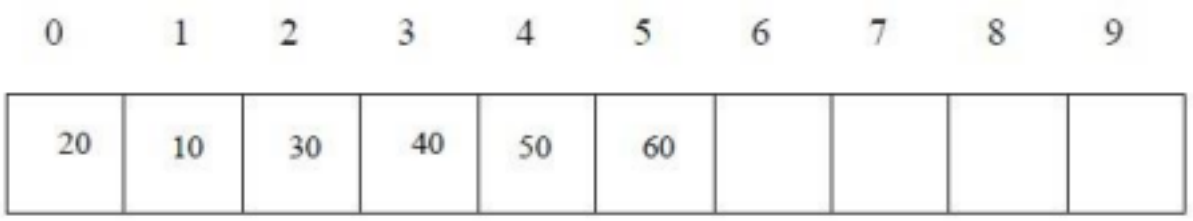
1. Array Implementation

2. Linked List Implementation

ARRAY IMPLEMENTATION OF LIST ADT

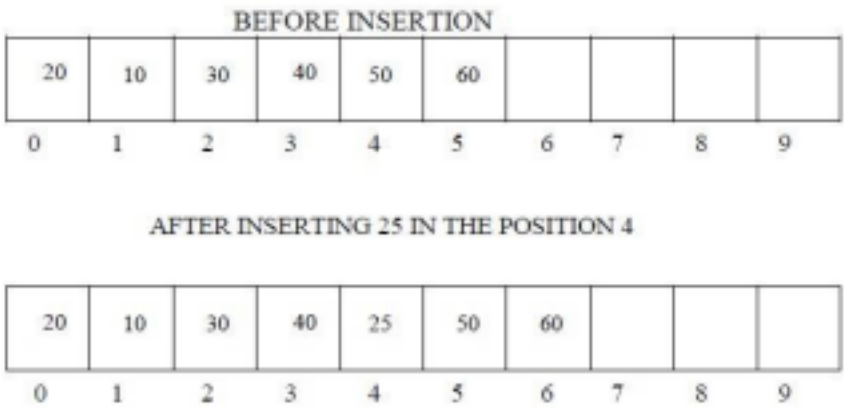
• Array is a collection of data stored in a consecutive memory location. • Insertion and Deletion operation are expensive as it requires more data

movement



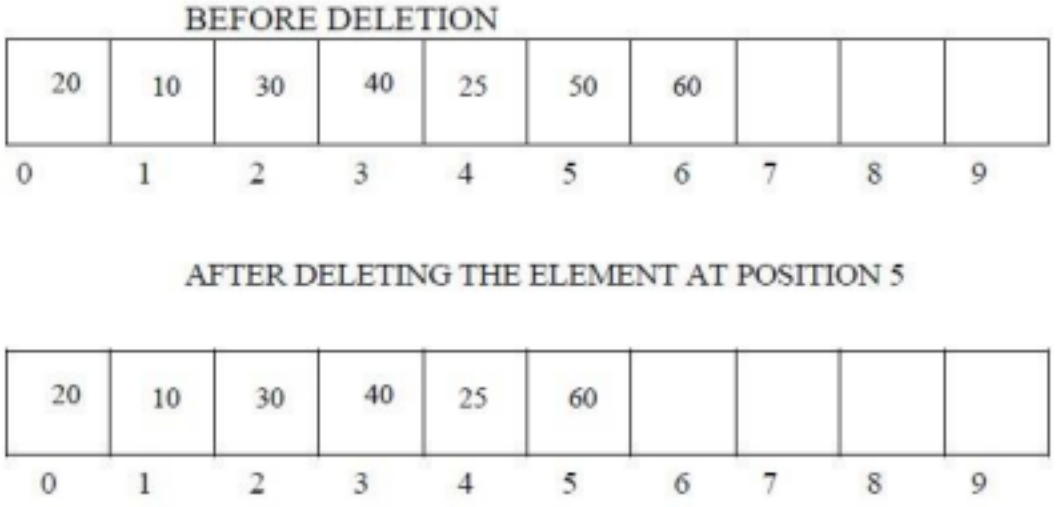
Insertion

- Insertion refers to the operation of adding another element to the list at the specified position.
  - When we insert an element into a position, the elements from that particular position to the last element are move forward one position.



Deletion

- Deletion refers to the operation of removing another element from the list at the specified position.
- When we delete an element from a position, the elements after that position are move backward one position.



Searching

Searching is a process of finding the position of an element

Traverse or Display

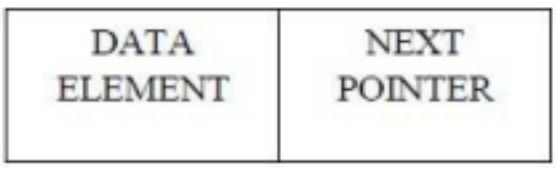
Traverse means print the elements in the list

Drawbacks in arrays:

- Has a fixed size.
- Data must be shifted during insertions and deletions.

LINKED LIST IMPLEMENTATION OF LIST ADT

- Linked list consists of series of nodes.
- Each node contains the element and a pointer to its successor node.
- The pointer of the last node is NULL.



Reason for Linked List

- While declaring arrays, we have to specify the size of the array, which will restrict the number of elements that the array can store.

Advantages of using linked list

- A linked list does not store its elements in consecutive memory locations and the user can add any number of elements to it.
- However, unlike an array, a linked list does not allow random access of data. Elements in a linked list can be accessed only in a sequential manner.

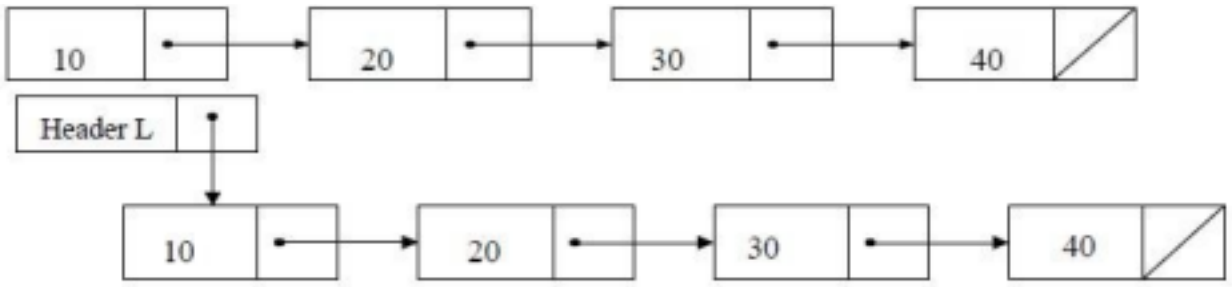
Types of linked list

1. Singly Linked List 2. Doubly Linked List 3. Circular Linked List.

1.6 SINGLY LINKED LIST

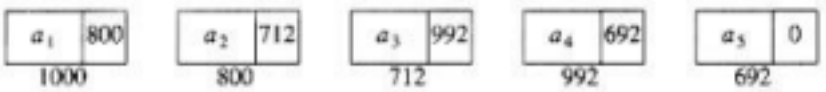
A singly linked list is a linked list in which each node contains only one link field pointing to the next node in the list.

Singly linked list



Header node points to the first node in the list

Linked list with actual pointer values



NODE DECLARATION

```
class Node:
 def __init__(self, data):
 # Initialize a new node with data and next pointer
 self.data = data


 self.next = None
```

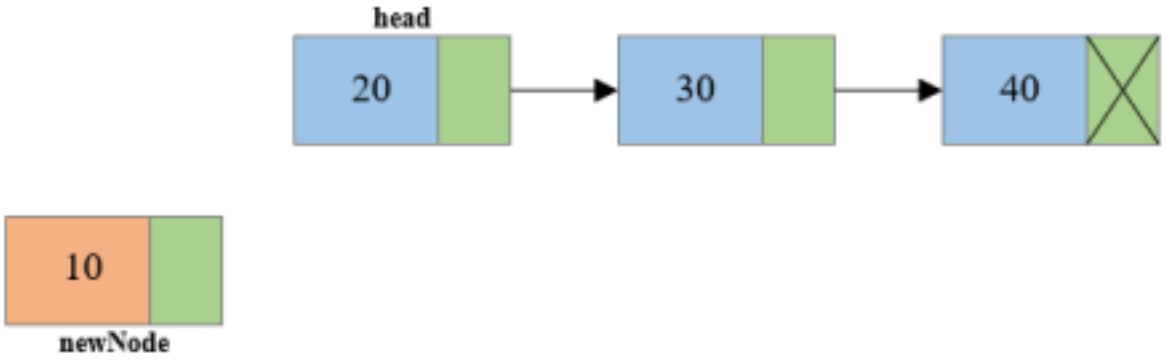
INSERTION:

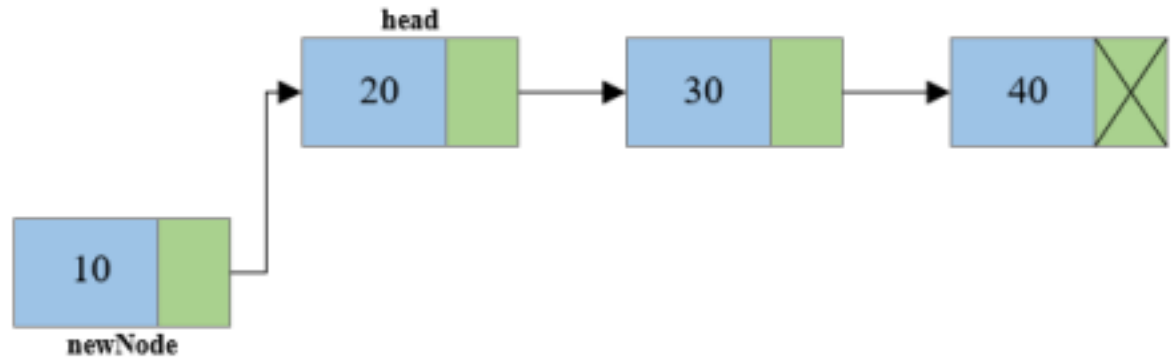
Overflow is a condition that occurs when no free memory cell is present in the system. When this condition occurs, the program must give an appropriate message. Insertion at the Beginning of the linked list:

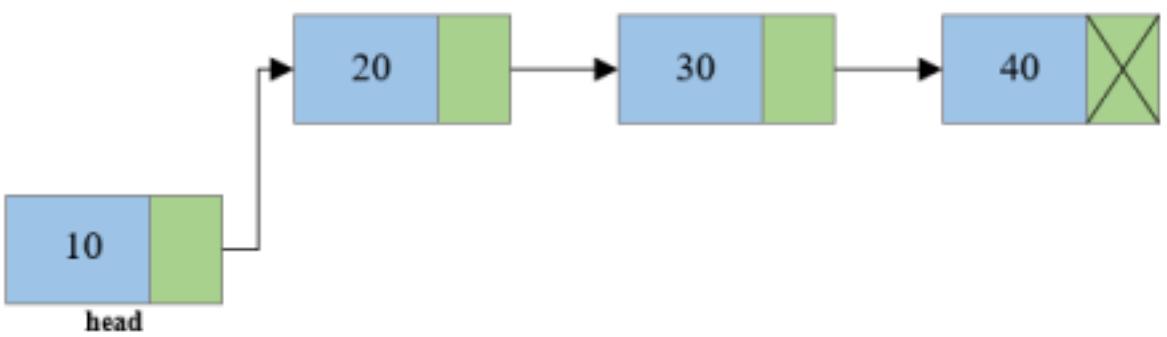
To insert a node at the beginning of a singly linked list in Python, you need to follow these steps:

• Create a new node with the given data.

• Set the "next" pointer of the new node to point to the current head of the list. • Update the head of the list to point to the new node.







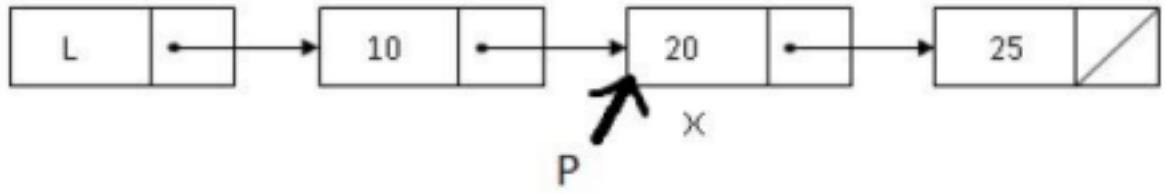

```
def insert_at_beginning(head, data):
 # Insert a new node at the beginning of the linked list
 new_node = Node(data)
 new_node.next = head
 return new_node
```

SEARCHING:

• Start from the first node and compare the data part with element to be searched

• If it is present then the corresponding position is returned.

target=20




```
def search(head, target):
 # Search for a target value in the linked list
 current = head
 position = 0
 while current:
 if current.data == target:
 print(f"Value {target} found at position {position}")
 return
 current = current.next
 position += 1
 print(f"Value {target} not found in the list.")
```

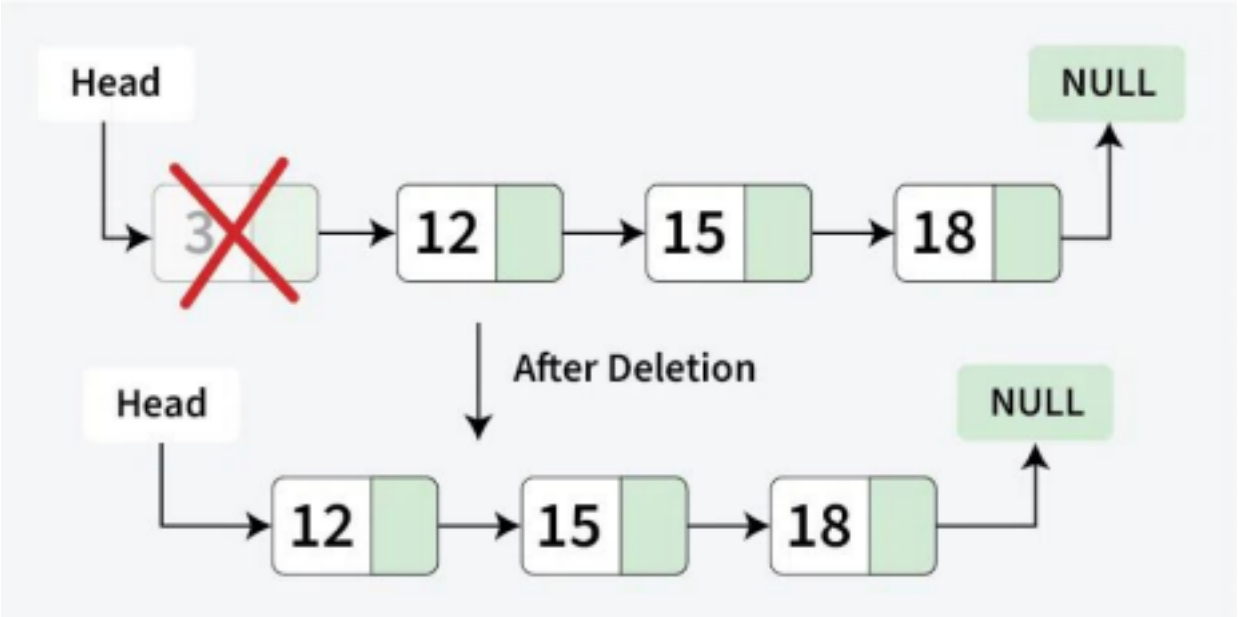
DELETION:

To delete a node from the beginning of a singly linked list in Python, you need to follow these steps:

- If the list is empty (head is None), there is nothing to delete.
- Update the head of the list to point to the next node (if any).

```
def delete_at_beginning(head):
 # Delete the node at the beginning of the linked list
 if head is None:
 print("Error: Singly linked list is empty")
 return None

 new_head = head.next
 del head
```

return new_head



TRAVERSE OR DISPLAY:

Traversal in a linked list means visiting each node and performing operations like printing or processing data.

Step-by-step approach:

- Initialize a pointer (current) to the head of the list.
- Loop through the list using a while loop until current becomes NULL. Process each node (e.g., print its data).
- Move to the next node by updating current = current->next.

```
def traverse(head):
 # Traverse the linked list and print its elements
 current = head
 while current:
 print(current.data, end=" -> ")
 current = current.next
 print("None")
```

ADVANTAGES OF SINGLY LINKED LIST

1) Dynamic memory allocation avoids wastage of memory.

2) Insertion and deletion of values is easier as compared to array

DISADVANTAGES OF SINGLY LINLED LIST

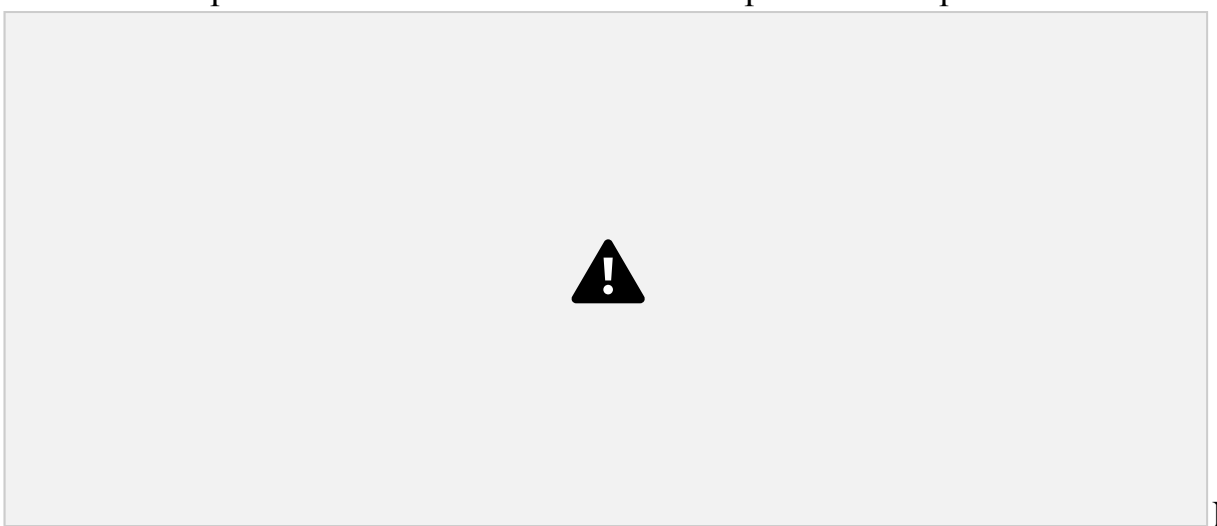
1) Nodes can only be accessed sequentially.

2) Binary search algorithm cannot be implemented.

3) Only forward traversal is possible

1.7 DOUBLY LINKED LIST

- A Doubly linked list is a linked list in which each node has three fields namely data field, forward link (FLINK) and Backward Link (BLINK).
- FLINK points to the successor and BLINK points to the predecessor.



Node declaration

class Node**:**

def **__init__(self, next**=None**, prev**=None**, data**=None**):**

 # reference to next node in DLL

self.next = next

 # reference to previous node in DLL

self.prev = prev

self.data = data

INSERTION:

1. Obtain space for new node.

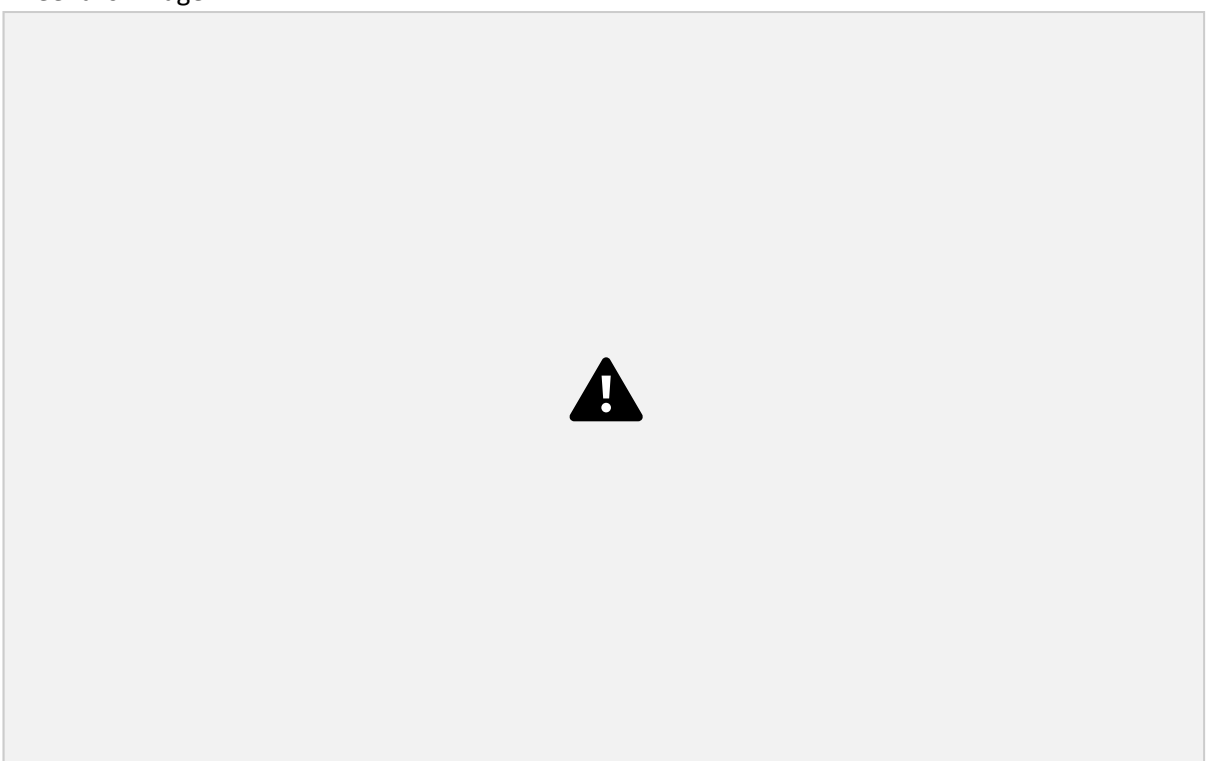
2. Assign data to the data field of the new node.

3. Assume current node P as the node after which we wish to insert.

4. Set the FLINK field of the new node to the successor node

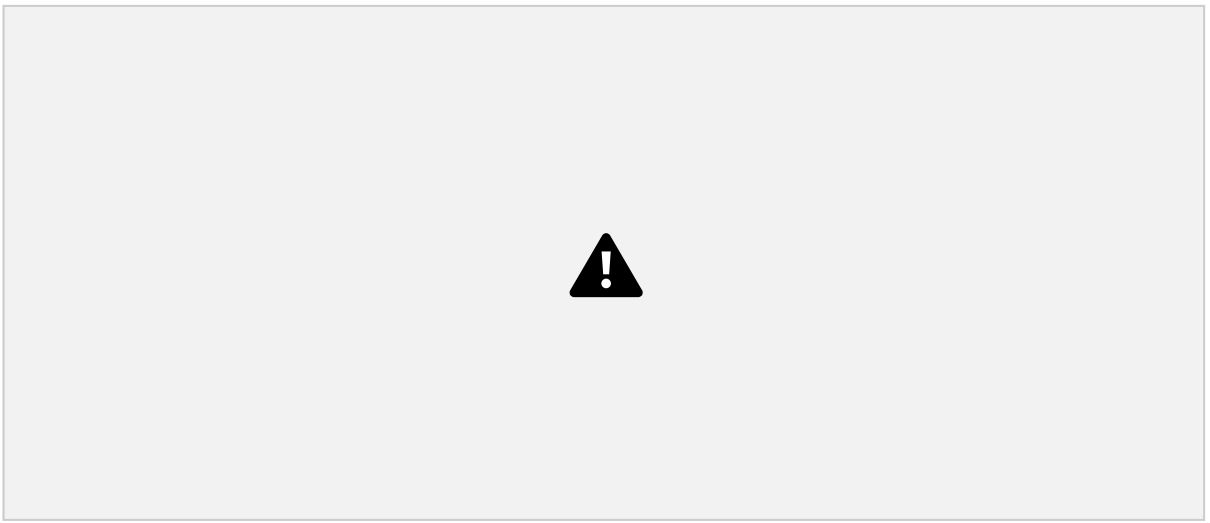
5. Set the FLINK of current node and BLINK of successor node to newnode.

6. Set the BLINK of newnode to current node



DELETION:

1. Find the node to be deleted and assume it as current node P 2. Set the FLINK of predecessor node as the FLINK of current node 3. Set the BLINK of successor node as the BLINK of the current node 4. Free the memory used by current node



SEARCHING:

- Start from the first node and compare the data part with element to be searched
- If it is present then the corresponding position is returned.

Procedure

```
def insert_at_beginning(head, data):
 new_node = Node(data)
 new_node.next = head
 if head:
 head.prev = new_node
 return new_node


def display(head):
 current = head
 while current:
 print(current.data, end=" <-> ")
 current = current.next
 print("None")


def delete_at_beginning(head):
 # Delete the first node from the beginning of the doubly linked list
if head is None:
```

```
print("Doubly linked list is empty")
return None

if head.next is None:
return None

new_head = head.next
new_head.prev = None
del head
return new_head
```

Advantages

- Deletion operation is easier.
- Finding the predecessor & successor of a node is easier.

Disadvantages

More memory space is required than singly linked list, since it has two pointers.

1.8 CIRCULAR LINKED LIST

Circular linked list is similar to normal linked list except that the last node contains a pointer to the first node of the list.

NODE DECLARATION

```
class Node:
 def __init__(self, data=None):
 # Data stored in the node
 self.data = data
 # Reference to the next node in the circular linked list
 self.next = None
```

Traversal of Circular Linked List in Python:

- Traversing a circular linked list involves visiting each node of the list starting from the head node and continuing until the head node is encountered again. def

```
traverse(self):
 # Traverse and display the elements of the circular linked list  if
not self.head:
 print("Circular Linked List is empty")
 return

 current = self.head
 while True:
 print(current.data, end=" -> ")
 current = current.next
 if current == self.head:
 # Break the loop when we reach the head again
 break
```


INSERTION

To insert a node at the beginning of a circular linked list, you need to follow these steps:

- Create a new node with the given data.
- If the list is empty, make the new node the head and point it to itself.
- Otherwise, set the next pointer of the new node to point to the current head.
- Update the head to point to the new node.
- Update the next pointer of the last node to point to the new head (to maintain the circular structure).

```
def insert_at_beginning(self, data):
 # Insert a new node with data at the beginning of the linked list
new_node = Node(data)
 if not self.head:
 self.head = new_node
 new_node.next = self.head
 else:
 new_node.next = self.head
```

```
temp = self.head
while temp.next != self.head:
temp = temp.next
temp.next = new_node
self.head = new_node
```

DELETION

To delete the node at the beginning of a circular linked list in Python, you need to follow these steps:

- Check if the list is empty. If it is empty, there is nothing to delete. • If the list has only one node, set the head to None to delete the node. • Otherwise, find the last node of the list (the node whose next pointer points to the head).
- Update the next pointer of the last node to point to the second node (head's next).
- Update the head to point to the second node.
- ~~Optionally, free the memory allocated to the deleted node.~~

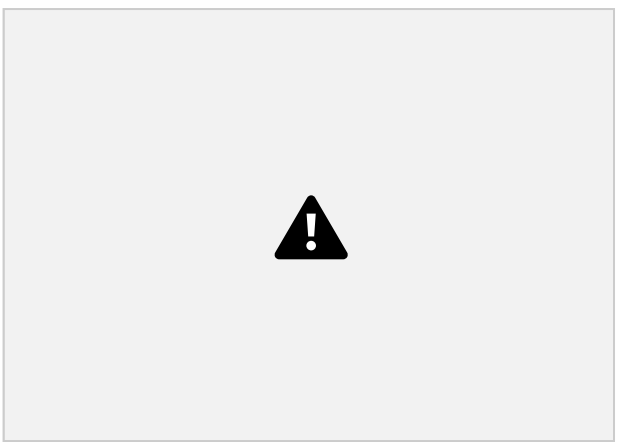

```
def delete_at_beginning(self):
if not self.head:
print("Circular Linked List is empty")
return

# If there is only one node in the list
if self.head.next == self.head:
self.head = None
return
current = self.head
while current.next != self.head:
current = current.next



# Update the last node to point to the second node
current.next = self.head.next
```

```
# Update the head to point to the second node
self.head = self.head.next
```

1.9 THE STACK ADT

Stack Model :

- A stack is a linear data structure which follows Last In First Out (LIFO) principle.
- Insertion and deletion occur at only one end called the Top.

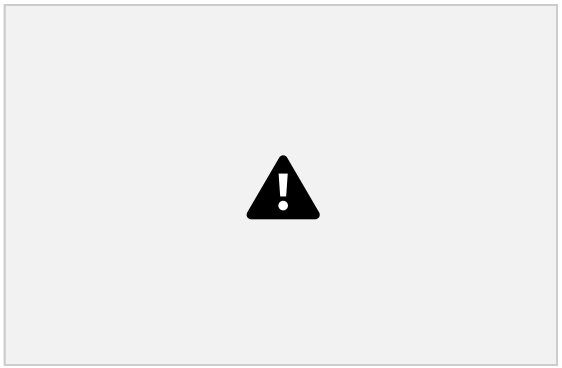


Examples: Pile of coins., a stack of trays in cafeteria

OPERATIONS ON STACK

The fundamental operations performed on a stack are

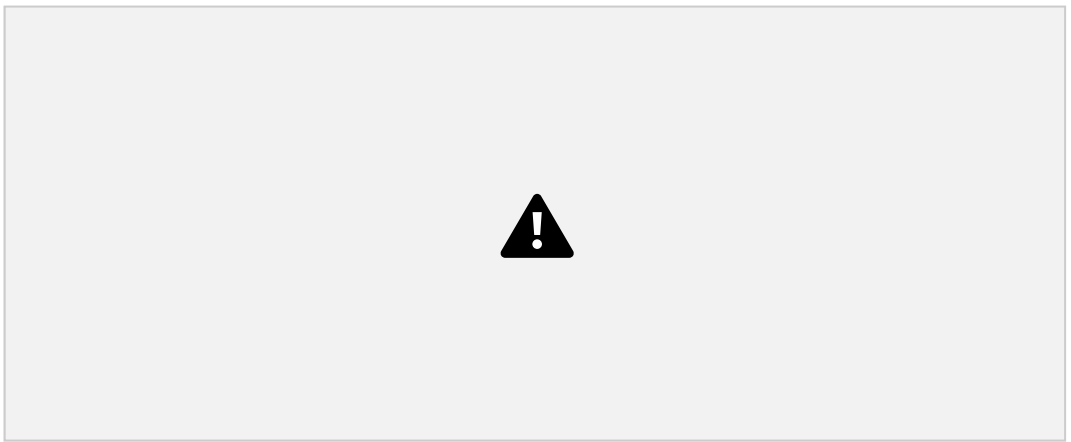
1. Push
2. Pop



1. PUSH :

• The process of inserting a new element to the top of the stack. •

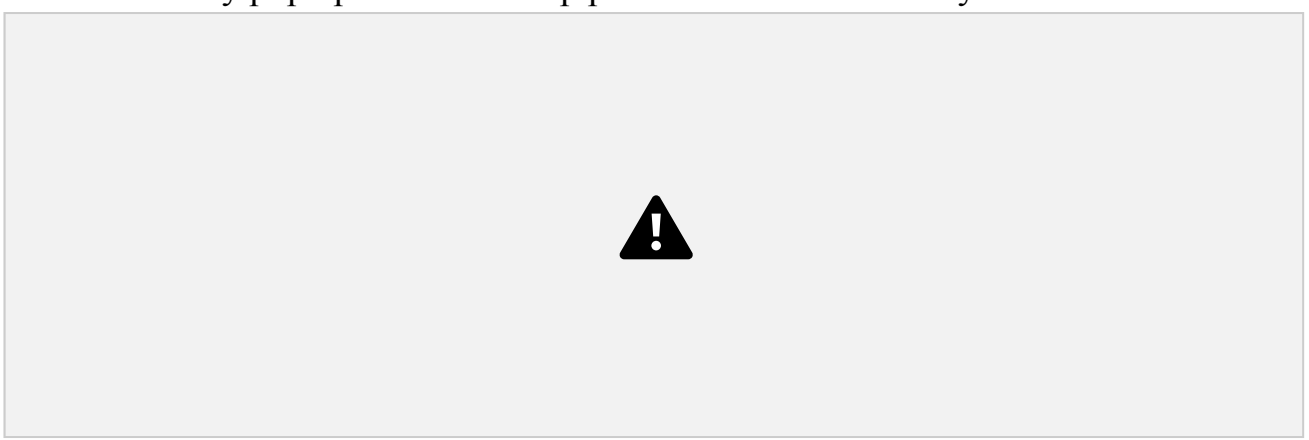
For every push operation the Top is incremented by 1.



2. POP :

• The process of deleting an element from the top of stack is called pop operation.

• After every pop operation the Top pointer is decremented by 1.



Conditions

Overflow

Attempt to insert an element when the stack is full is said to be

overflow. **Underflow**

Attempt to delete an element, when the stack is empty is said to be underflow **IMPLEMENTATION OF STACK ADT**

Stack can be implemented using

1. Array 2. Linked List .

Array Implementation

- Each stack is associated with a Top pointer.
- Top = -1 for an empty stack.
- To insert an element(PUSH),
    - Top is incremented by 1.
    - Store the element in position Top. (stack[Top]=element).
    - Push on a full stack is overflow.
- To delete an element(POP),
    - The stack [Top] value is returned.
    - Top pointer is decremented by 1.
    - Pop on an empty stack is underflow.

Stack using Linked List

- The major problem with the stack implemented using array is, **it works only for fixed number of data values.** That means the amount of data must be specified at the beginning of the implementation itself.
- In linked list implementation of a stack, every new element is inserted as 'top' element.
- That means every newly inserted element is pointed by 'top'. Whenever we want to remove an element from the stack, simply remove the node which is pointed by 'top'

OPERATIONS

Step 1: Define a 'Node' structure with two member's data and next. Step 2: Define a Node pointer 'top' and set it to NULL.

push(value) - Inserting an element into the Stack

We can use the following steps to insert a new node into the stack... Step 1: Create a newNode with given value.

Step 2: Check whether stack is Empty (top == NULL)

Step 3: If it is Empty, then set newNode → next = NULL.

Step 4: If it is Not Empty, then set newNode → next = top.

Step 5: Finally, set top = newNode.

```
def push(self, data):
new_node = Node(data)
new_node.next = self.top
self.top = new_node

print(f"Pushed {data} into stack")
```


pop() - Deleting an Element from a Stack

We can use the following steps to delete a node from the stack... Step 1: Check whether stack is Empty (top == NULL).

Step 2: If it is Empty, then display "Stack is Empty!!! Deletion is not possible!!!" and terminate the function

Step 3: If it is Not Empty, then define a Node pointer 'temp' and set it to 'top'. Step 4: Then set 'top = top → next'.

Step 5: Finally, delete 'temp' (free(temp)).

```
def pop(self):
if self.top is None:
print("Stack Underflow (empty stack)")
return None
```

```
popped = self.top.data
self.top = self.top.next
print(f"Popped {popped} from stack")
return popped
```

1.10 EXPRESSION EVALUATION

Different Types of Notations to Represent Arithmetic Expressions: There are 3 different ways of representing the algebraic expression. * INFIX EXPRESSION

* POSTFIX EXPRESSION
* PREFIX EXPRESSION

INFIX EXPRESSION

• Operator appears between the two operands.

For example : - A / B + C

POSTFIX EXPRESSION

• Operator appears after the two operands

• It is also called reverse polish notation.

For example : - AB / C + [Infix: ((A/B) + C)]

PREFIX EXPRESSION

• Operator appears before the two operands.

• It is also called as polish notation.

For example : - + / ABC [Infix : ((A/B) + C)]

Infix	Prefix	Postfix
A+B	+AB	AB+
A+B-C	-+ABC	AB+C-
(A+B)*C-D	-*+ABCD	AB+C*D

EVALUATING ARITHMETIC EXPRESSION

- To evaluate an arithmetic expressions, first convert the given infix expression to postfix expression and then evaluate the postfix expression using stack.

Algorithm:

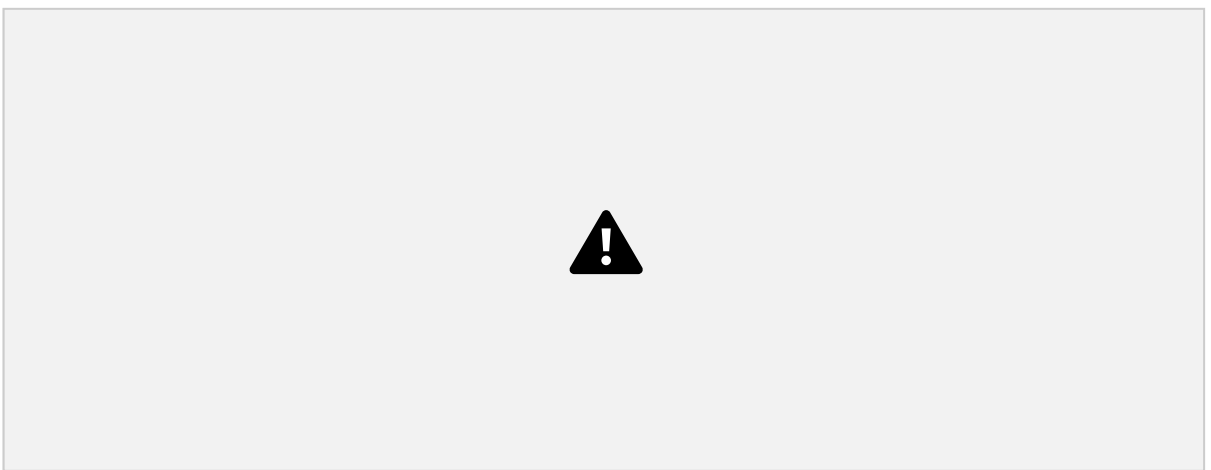
Read the postfix expression one character at a time until it encounters the delimiter „ “#”.

Step 1 : - If the character is an operand, push its associated value onto the stack.

Step 2 : - If the character is an operator, POP two values from the stack, apply the operator to them and push the result onto the stack. The first pop is the second operand and the second pop is the first operand.

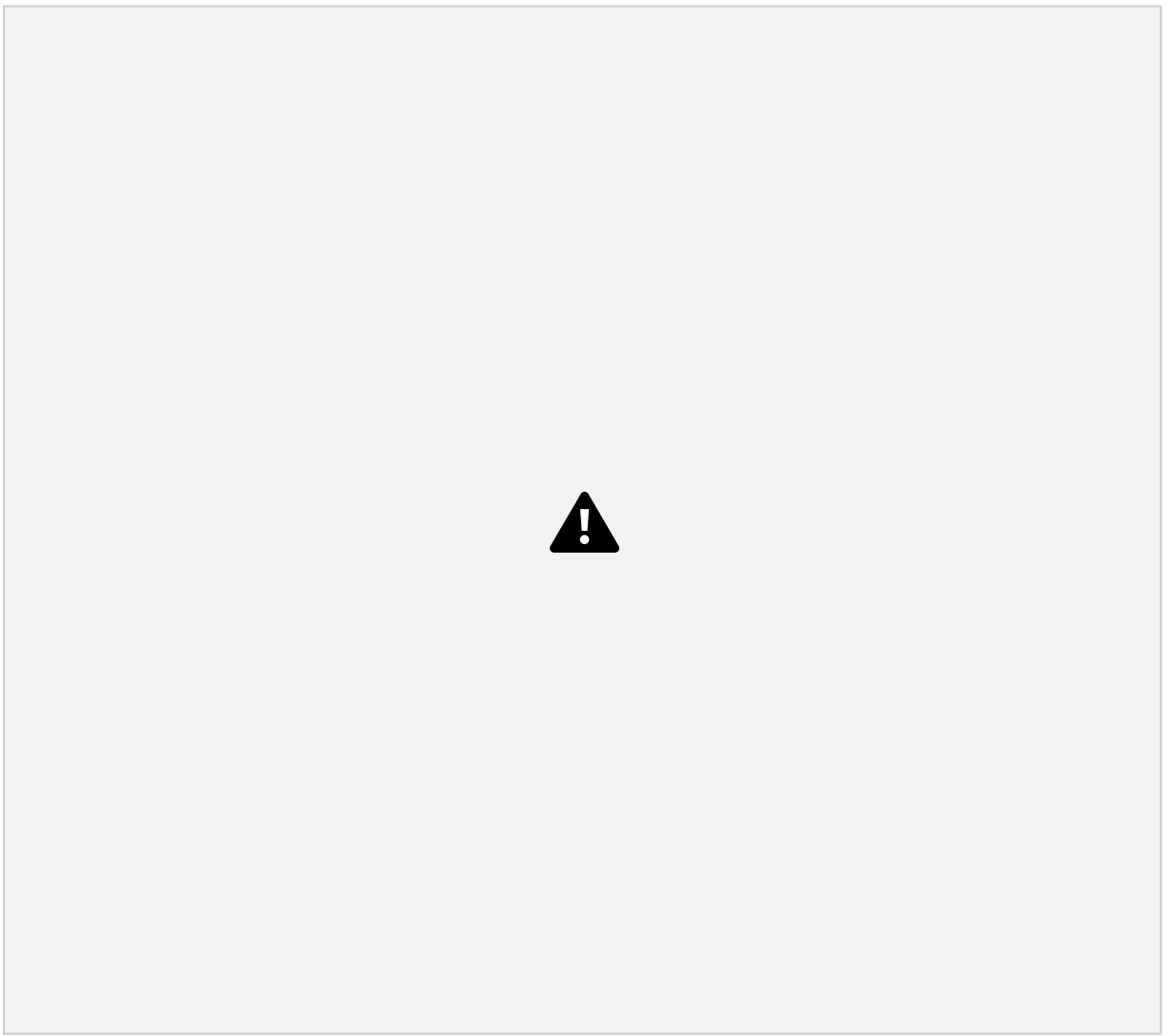
Example 1:

Post fix Expression : AB* CDE/-+ #

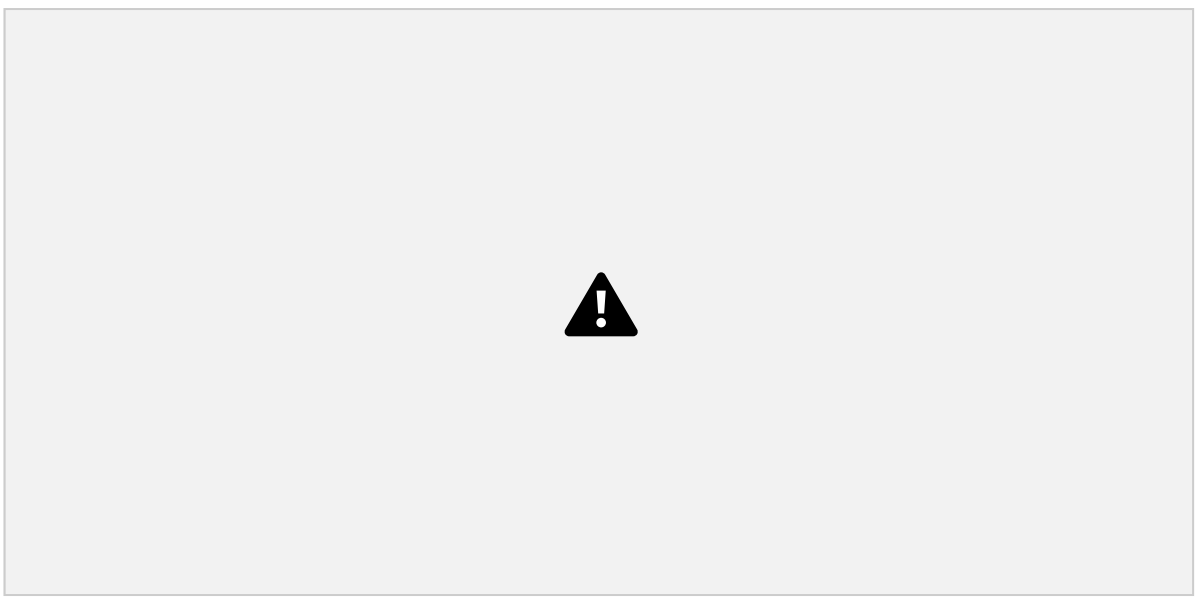


Solution:

From the given expression read one char



Answer= 21



Example 2: **Input**: S = "123+*8-"

Solution: -3

1.11 SYNTAX CHECKING (BALANCED PARENTHESES) • Stacks can be used to check whether the given expression has balanced symbols or not. The algorithm is very much useful in compilers.

• Every right brace, bracket, and parenthesis must correspond to their left counterparts. The sequence [()] is legal, but [(]) is wrong. The simple algorithm uses a stack and is as follows.

Steps:

Read one character at a time and repeat steps 1 and 2 until it encounters the delimiter '#'.

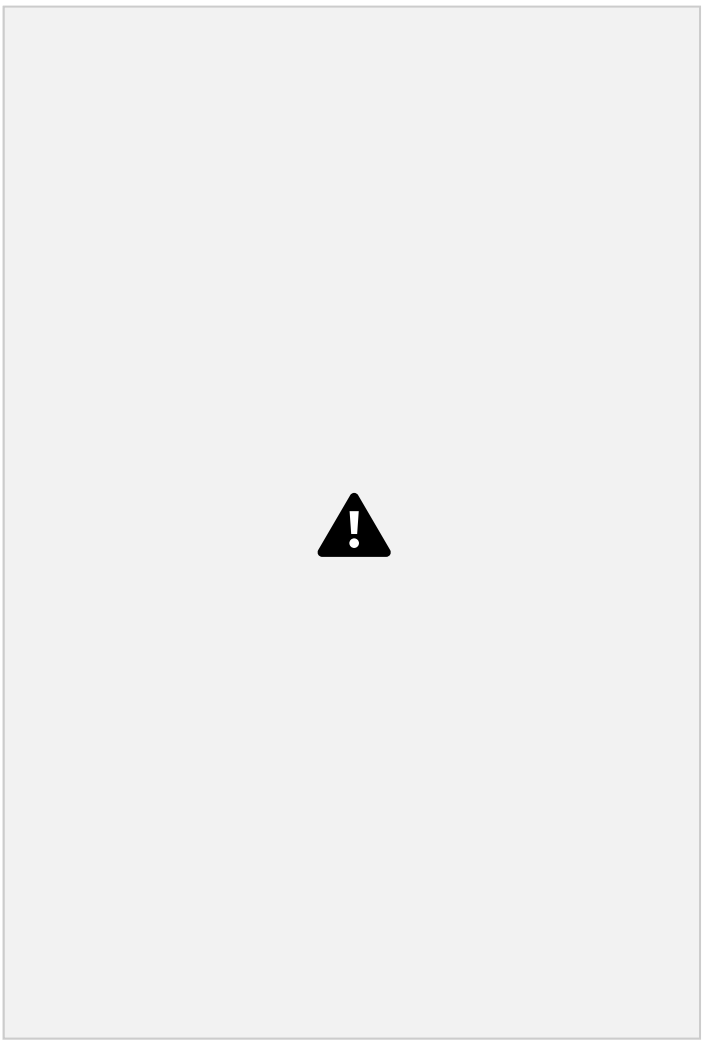
Step 1 : If the character is an opening symbol, push it onto the stack.

Step 2 : If the character is a closing symbol,

2.1: If it has corresponding opening symbol in the stack, POP it from the stack. Otherwise, report an error "Unbalanced".

2.2: If the stack is empty report an error "Unbalanced".

Step 3 : At the end, if the stack is not empty, report an error "Unbalanced". Otherwise, report "Balanced".



Example 2: ((A+B) + [C-D]] **Not Balanced**

1.12 QUEUE ADT

Queue Model

A **Queue** is a linear data structure which follows **First In First Out (FIFO)** principle, in which **insertion** is performed at **rear** end and **deletion** is performed at **front** end.

Example: Waiting Line in Reservation Counter, queue in a cinema ticket counter, Scheduling of jobs in computer

OPERATIONS ON QUEUE

The fundamental operations performed on queue are

1. Enqueue
2. Dequeue

ENQUEUE:

- The process of inserting an element in the queue.

DEQUEUE:

- The process of deleting an element from the queue.

EXCEPTION CONDITIONS

Overflow: Attempt to insert an element, when the queue is full is said to be overflow condition.

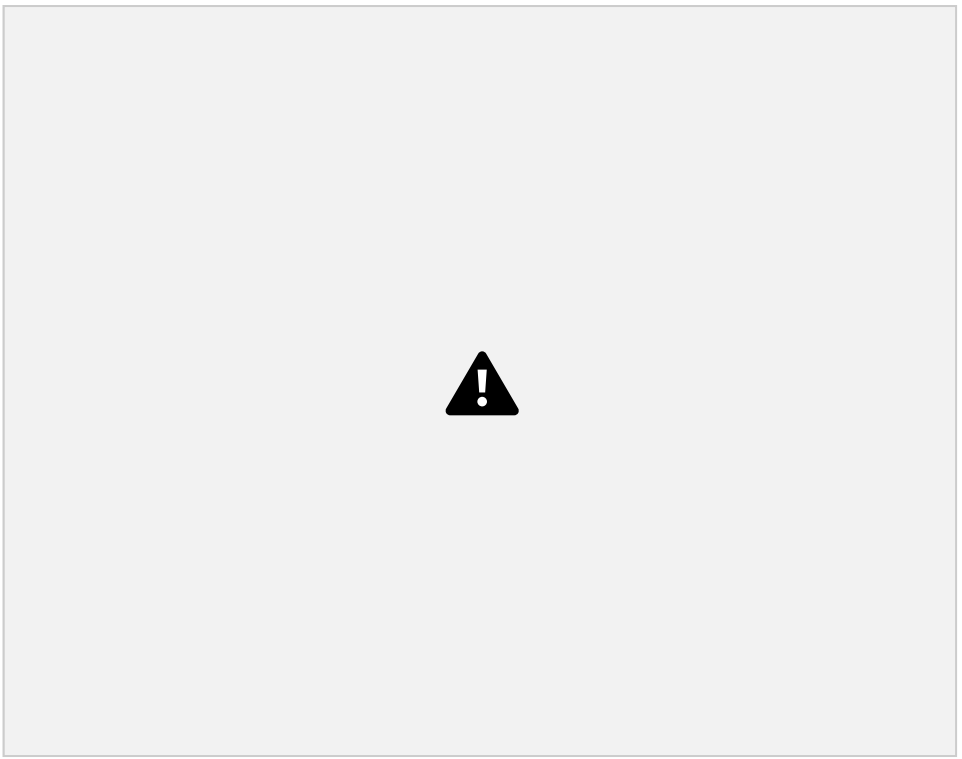
Underflow: Attempt to delete an element from the queue, when the queue is empty is said to be underflow.

IMPLEMENTATION OF QUEUE

Queue can be implemented using array and linked list.

ARRAY IMPLEMENTATION

- In array implementation queue Q is associated with two pointers namely Rear and Front.
- For an empty queue Front = -1 and Rear = -1
- To insert an element X onto the queue Q, the Rear is incremented by 1 and then set Q [Rear] = X
- To delete an element, the Q [front] is returned and the Front is incremented by 1.



Linked List Implementation of Queue

• Enqueue operation is performed at the end of the list (Rear end) • Dequeue operation is performed at the front of the list (Front end)

```
def enqueue(self, x):
 temp = Node(x)

 if self.rear is None:
 self.front = temp
 self.rear = temp
 else:
 self.rear.next = temp
 self.rear = temp

 self.size += 1
```



```
def dequeue(self):
```

```
if self.front is None:
return None

# Store the front node's value
res = self.front.key

# Move the front pointer to the next node
self.front = self.front.next

# If the queue becomes empty, set rear to None
if self.front is None:
self.rear = None

self.size -= 1
return res
```

TYPES OF QUEUE

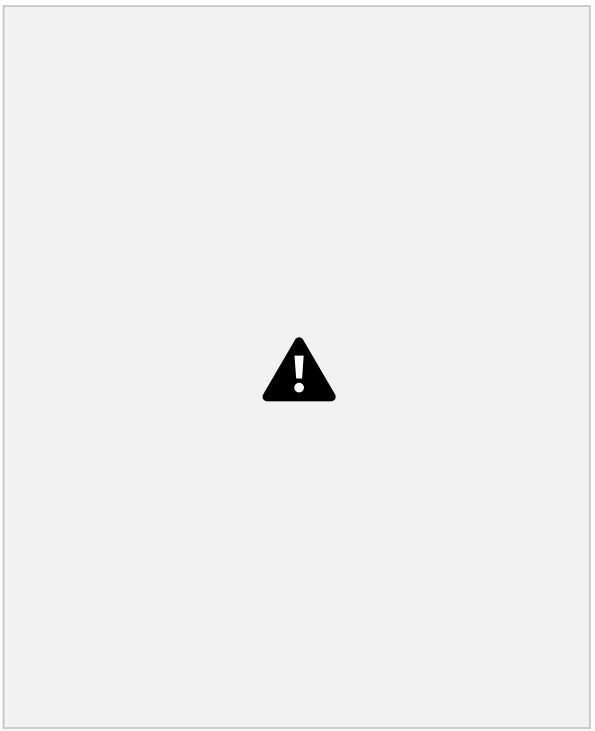
There are four different types of queues:

- Simple Queue
- Circular Queue
- Priority Queue
- Double Ended Queue

1.13 CIRCULAR QUEUE

Circular Queue Model

In Circular Queue, elements are arranged in circular fashion. So the first element comes just after the last element. If the last location of the queue is full, Insertion of a new element is performed at the very first location of the queue



Advantage

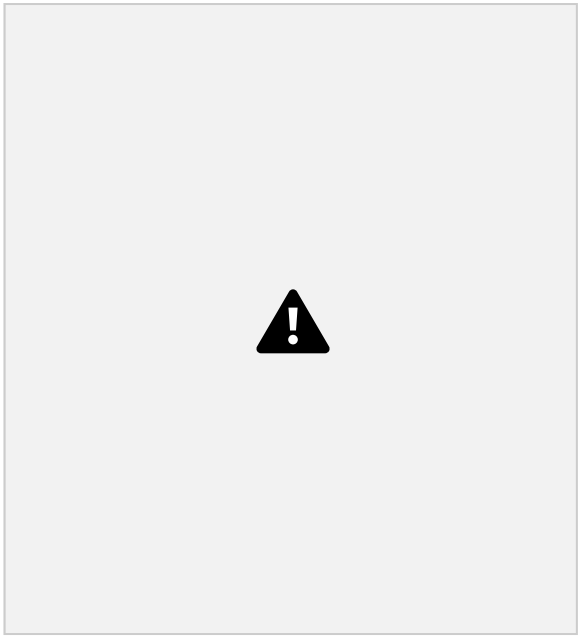
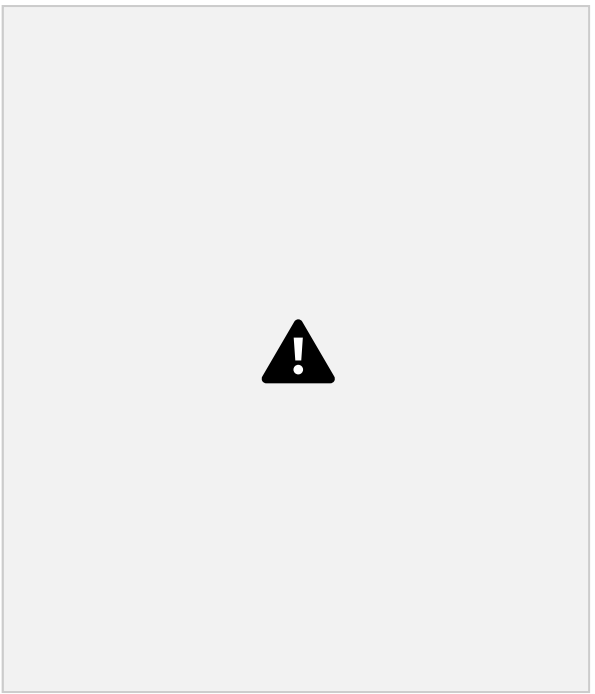
It overcomes the problem of unutilized space in linear queues, when it is implemented as arrays.

Circular Queue Operations

- To perform the insertion of an element to the queue, the position of the element is calculated by

  Rear = (Rear + 1) % Maxsize. and then set CQ [Rear] = value. • To perform the deletion,element is taken from Front

  Value = CQ [Front] and the position of the Front printer is calculated by Front = (Front + 1) % maxsize.

```
# Enqueue operation
def enqueue(self, data):
new_node = Node(data)
if self.is_empty():
self.front = new_node
self.rear = new_node
new_node.next = self.front # circular link
else:
self.rear.next = new_node
self.rear = new_node
self.rear.next = self.front # maintain circular link

# Dequeue operation
def dequeue(self):
if self.is_empty():
print("Queue is Empty!")
return None
```



```
value = self.front.data
```

```
if self.front == self.rear: # only one element
self.front = None
self.rear = None
else:
self.front = self.front.next
self.rear.next = self.front # maintain circular link
return value
```

1.14 PRIORITY QUEUE

A priority queue is a special type of queue in which each element is associated with a priority and is served according to its priority. If elements with the same priority occur, they are served according to their order in the queue. Insertion occurs based on the arrival of the values and removal occurs based on priority

1.15 APPLICATIONS IN SCHEDULING

A queue works on FIFO (First In First Out). Scheduling often involves tasks, processes, or jobs waiting for some resource (CPU, printer, disk, etc.). So, queues help in managing order, fairness, and resource sharing.

Circular Queue → used in Round Robin CPU scheduling.

Priority Queue → used in priority-based scheduling (OS, printers, networks). Simple FIFO Queue → used in FCFS systems (disk, customer service). **1.16 RECURSION VS ITERATION**

- Recursion is a technique where a function calls itself directly or indirectly to solve a problem. It works by dividing a problem into smaller subproblems until a base case is reached.
- Iteration means repeating a set of instructions (loop) until a condition becomes false.
- How Recursion Uses Stack

Resursion

Each recursive call is pushed onto the call stack.

Stack frame stores:

Function parameters

Local variables

Return address

When base case is reached, functions return and stack frames are popped one by one. **Iteration**

Iteration (loops) does not create new stack frames. Only one function call stays in stack. Loop variables update inside the same frame.

1.17 REAL-WORLD USE CASES: UNDO-REDO SYSTEMS, AND TASK QUEUE MANAGEMENT

1. Undo–Redo Systems

- Used in text editors, graphic software, IDEs, etc.

Undo → reverse the most recent action.

Redo → reapply an action that was undone.

Data Structures Used:

Two Stacks

`Undo stack`: Stores performed actions.

`Redo stack`: Stores undone actions.

Workflow:

1. Perform an action → push it onto Undo stack, clear Redo stack.

2. Undo an action → pop from Undo stack → push onto Redo stack. 3. Redo an action → pop from Redo stack → push onto Undo stack.

Example (Typing “ABC”):

Undo stack: [A, B, C]

Redo stack: []

Undo → remove "C"

Undo stack: [A, B]

Redo stack: [C]

Redo → restore "C"

Undo stack: [A, B, C]

Redo stack: []

```



2. Task Queue Management

- Used in operating systems, servers, and applications to manage jobs waiting for
```

resources.

How it works:

- Tasks are added to a queue (FIFO order).
- Scheduler picks tasks from the queue and executes them.
- Ensures fairness and efficient resource utilization.

Examples:

1. CPU Scheduling
   - Processes waiting for CPU are kept in a ready queue.
   - Algorithms: FCFS (queue), Round Robin (circular queue), Priority Queue.
2. Print Job Scheduling
   - Print requests are stored in a print queue.
   - Printer serves jobs in order, unless priority overrides.
3. Message Queues in Systems
   - Asynchronous communication (e.g., email server, RabbitMQ, Kafka). • Producers add tasks → Consumers process them.